

An Approach for Peer-to-Peer Federated Learning

Tobias Wink

Karlsruhe University of Applied Sciences

Karlsruhe, Germany

tobias.wink@h-ka.de

Zoltan Nochta

Karlsruhe University of Applied Sciences

Karlsruhe, Germany

zoltan.nochta@h-ka.de

Abstract—We present a novel approach for the collaborative training of neural network models in decentralized federated environments. In the iterative process a group of autonomous peers run multiple training rounds to train a common model. Thereby, participants perform all model training steps locally, such as stochastic gradient descent optimization, using their private, *e.g.* mission-critical, training datasets. Based on locally updated models, participants can jointly determine a common model by averaging all associated model weights without sharing the actual weight values. For this purpose we introduce a simple *n*-out-of-*n* secret sharing schema and an algorithm to calculate average values in a peer-to-peer manner. Our experimental results with deep neural networks on well-known sample datasets prove the generic applicability of the approach, with regard to model quality parameters. Since there is no need to involve a central service provider in model training, the approach can help establish trustworthy collaboration platforms for businesses with high security and data protection requirements.

Index Terms—Machine Learning, Federated Learning, Security, Privacy

I. INTRODUCTION

Machine learning (ML) technologies, especially artificial neural networks, have developed rapidly in recent years and are used in a broad range of applications meanwhile. In a typical technical setup the same entity owns and controls sample datasets that it uses to train and test ML models. In an example use case a bank could create and train a deep neural network (DNN) to detect and potentially abort suspicious payment transactions. Locally available training datasets, *i.e.*, data that the bank has gathered about past transactions, might be of limited size depending on the bank's actual market share, number of customers, etc. Training data might also lack of relevant features, as it might not yet contain enough traces of newer attacks, or illicit activities of new, malicious customers. To improve the accuracy of fraud detection and take the right countermeasures in their systems, banks (as data owners) could cooperate with each other and jointly train a superior DNN model by putting together all available training data in a central data pool. In reality, a bank would be rather reluctant to share its business critical data with any third party, especially not with its competitors. In addition, regulatory hurdles, such as General Data Protection Regulation (GDPR), might also nip such a cooperation in the bud.

In such situations data owners could establish a federated learning (FL) system and collaboratively train a common

model without exposing training datasets to others. The resulting model should have a performance very close to a model that could be trained when pooling the data of all data owners, *e.g.*, in a centralized model training system [1].

Federated optimization (FO) [2] uses a central server to create and to step-wise refine a model without collecting training datasets of data owners. Instead, data owners independently train a previously distributed model and send their adjusted weight values to the server. The server aggregates received weights, creates an updated version of the common model, and distributes the new model to the participants.

Nevertheless, from security perspective the central server has to be treated as a trusted entity: It has to aggregate all received model weights in the correct way without discriminating any of the clients. Furthermore, it must not conduct malicious reverse engineering, such as Generative Adversarial Model INversion (GAMIN) [3], membership inference attacks [4], etc. using the received client-specific model weights neither. With the help of such emerging attacks the service provider could learn much about single clients' actual training data and reconstruct, *e.g.*, a bank's payment transactions.

Enhancements of FO by means of Differential Privacy (DP) [5] can help better protect client's data privacy [6, 7, 8]. These measures are in particular effective in systems having a very large population, *e.g.*, thousands or even millions, of clients.

Our approach can basically be seen as a peer-to-peer [9] variant of FO. Due to the relatively high communication and computation costs, it can help improve data secrecy and privacy protection especially in systems that serve smaller groups of collaborating data owners. Group members are able to collaboratively aggregate all corresponding ML model weights, and thus create a common ML model of a desired quality *without* involving a central server or third-party entity in the process.

For this purpose we introduce a lightweight *n*-out-of-*n* secret sharing schema and an algorithm to calculate average weight values in a peer-to-peer and privacy-preserving manner. During the joint computation of aggregated model weights all participants keep all training and test datasets (incl. labels) private, and, they do not exchange model weights neither. In accordance with [10] the underlying threat model assumes *semi-honest* (a.k.a. *honest-but-curious*) participants, who run the protocol honestly, but may try to learn as much as possible from the messages they receive from other parties.

Funded by the German Federal Ministry of Education and Research. Project name: KIWI, RefNr: 16KIS1142K

II. RELATED WORK

A. Federated Optimization

Federated optimization (FO) is a method to create and continuously improve a neural-network model based on distributed training data. It was developed at Google to train models with the help of a large number of mobile devices [2, 11, 12, 13, 14, 15] and is used today, *e.g.*, for keyboard prediction [16]. A schematic illustration of a training round in FO is shown in Figure 3. Upon starting a training round, the central server randomly determines a subset of available nodes. Each selected node trains its local copy of the previously deployed model using its own data and sends updated model weights to the server afterwards. The server computes average values from the weights it has actually received and updates his copy of the model accordingly. The updated model must finally be distributed to all nodes, at best before a new training round would begin. FO is in particular advantageous in scenarios with the following key characteristics:

- **Non-IID data:** Data stored on mobile devices is typically very much user-dependent, thus training data on a particular node might show user-specific feature distributions as well.
- **Unbalanced data:** The amount of training data may vary significantly between nodes. Some nodes might have a multiple of the data available compared to others.
- **Massive distribution:** The number of nodes (*e.g.*, millions), can significantly be higher than the amount of available training samples (*e.g.*, thousands).
- **Technical constraints:** Especially mobile devices may have limited network connectivity and battery capacity, leading to relatively low training frequency, for example, once per day.

In general, FO provides higher level of data privacy compared to many other distributed learning approaches shown in [17], because the information transmitted between nodes and the server is limited to updated weights only. In [18], the basic FO process was further extended by DP principles [19, 5]. The TensorFlow extension TensorFlow Privacy [20] enables a straightforward application of these principles in practical projects. An alternative approach, called Private FO [7] deals with DP for data stored on the central server as well as on client nodes. An approach for adaptive FO has been introduced in [21], allowing even the combined usage of different optimizers on nodes and/or the central server.

B. Further Distributed Model Training Approaches

Split learning (SL) [22, 23] is a means to train neural-network models without sending training data of client nodes to the central server. The basic idea is to split the model into a server and a node part. Before starting the training, the server sends the initial not yet trained model's node part to the nodes. A node runs the forward-pass using local training data. The resulting tensor is sent to the server, which uses it as input to train its model layers. During the backward pass the server performs back-propagation and sends the resulting gradient

tensor of its model part's first layer to the node. The node uses the received tensor for its local back-propagation steps and completes the training round. To run supervised model training, the node has to send its classification labels for each training data item to the server. NoPeekNN [24] reduces the risk of training data leakages. To run SL with multiple nodes, the node model must be shared between all involved nodes. SL might significantly slow down a model training pipeline, because only one node can conduct model training at the same time, and a huge amount of data (*e.g.*, millions of weights) has to be exchanged between nodes and the server in each forward- and backward-pass, incl. required data (de-)serialization steps.

BrainTorrent [25] is a system to train a common model in a peer-to-peer manner. To start a training round, a participant P first asks all other peers whether they meanwhile updated, *i.e.*, re-trained their local models. Peers that possess newer model versions send their weights to P together with the respective training sample's size. P then updates its local model weights by computing a weighted average of its own (old) model weights and the received (supposed newer) model weights. The system does not rely on a central service or data pool. Nevertheless, a semi-honest participant might successfully obtain training data of (all) other peers by reverse engineering the received weights. A participant could furthermore act in many ways malicious, without being detected. For example, she or he might refuse giving honest answers regarding to its current model version number, and thus disproportionately profit from other peers' training efforts.

[26] introduces a fully decentralized and asynchronous algorithm for peer-to-peer machine learning under privacy constraints. The focus is on personalized models, *i.e.* not on the creation of a common model. The communication takes place within neighborhoods of nodes that either have similar datasets and/or training goals in order to limit the overall effort. The nodes use Differential Privacy [5] to preserve privacy and transfer the model after a training round via broadcast. In each training round, the latest available versions of the neighbors' models influence a node's own training. Due to the asynchronous process, however, older and meanwhile outdated models may also be included in some cases.

In [27] a framework is outlined to share and validate DNN model parameters based on distributed ledger technology. An entity, *e.g.*, a self-driving car, first trains a locally deployed model using its own locally obtained data. After completing the training, model parameters are added to the Blockchain, thus other users can verify the parameters' integrity before creating and offering new models. A tokenization mechanism supports active participants in gaining fair (monetary) benefits. Practical aspects, such as training performance using existing models and data, are not discussed. It also remains unclear, how the system could protect against phenomena, such as over-fitting (caused by very active participants), or "catastrophic forgetting" of features that are relevant for rather occasional participants, etc.

C. Secret Sharing

Secret sharing schemes [28, 29] serve as cryptographic primitives in several secure multi-party computation (MPC) protocols [10]. The aim is to divide and distribute a secret s among n parties in such a way that only the combined usage of k parts (possessed by k different entities) can help reconstruct the original secret. There are three basic families:

- $1 < k \leq n$: In these so called (k, n) threshold schemes the secret s is divided into n parts, but already k parts are sufficient to reconstruct the secret. Shamir's secret sharing [28] is one of the first published (k, n) threshold schemes.
- $k=n$: These schemes are also called n -out-of- n schemes, since all parts, each held by at least one of n parties, are required to reconstruct s . Additive secret sharing [30] is probably the simplest n -out-of- n secret sharing scheme. To split a secret number s into n shares, $n - 1$ field elements $s_1, \dots, s_{n-1}$ are chosen randomly, and s_n is defined as $s_n = s - \sum_{i=1}^{n-1} s_i$. Each of n parties then gets one of these shares. To reconstruct s , all s_i shares are summed up.
- $k=1$: It is the trivial case, since s can easily be distributed to any (all) of n parties turning s into a "shared secret".

III. CONCEPT

Decentralized peer-to-peer solutions have several potential advantages over systems that rely on some central system or service. These include greater autonomy and sovereignty of participants regarding process control and data management as well as reduced risk of outages due to server issues. Gaining such primarily technical advantages must not, of course, lead to functional limitations from users' point of view. Thus, the most basic requirement for a federated peer-to-peer ML system is that collaboratively trained models should have at least the same quality (prediction accuracy) as if a central server had been involved in the training.

Figure 1 shows an abstract overview of the iterative peer-to-peer model training process with three participants $Peer_1$, $Peer_2$ and $Peer_3$ as an example. In a first step, an arbitrary participant, say $Peer_1$ may create an instance of the ML model v_0 . Initially, all weights in v_0 can be random values. $Peer_1$ sends v_0 to all other participants as starting point to collaboratively train a common "global" model. In an alternative setup, participants may also retrieve or even buy v_0 from some third-party entity. In the first training round, each peer (including the initiator $Peer_1$) trains its own respective copy of v_0 locally. Local training steps are conducted by using local sample datasets only, which the given participant wants to keep secret. During local training, *e.g.*, using stochastic gradient descent (SGD) optimization, no interaction or data exchange between any of the parties is required. After successful completion of local training steps, each peer will possess a new, updated version of the original model v_0 . These updated model instances, in Figure 1 denoted as v'_0 , v''_0 and v'''_0 will differ from each other, because during the local training different

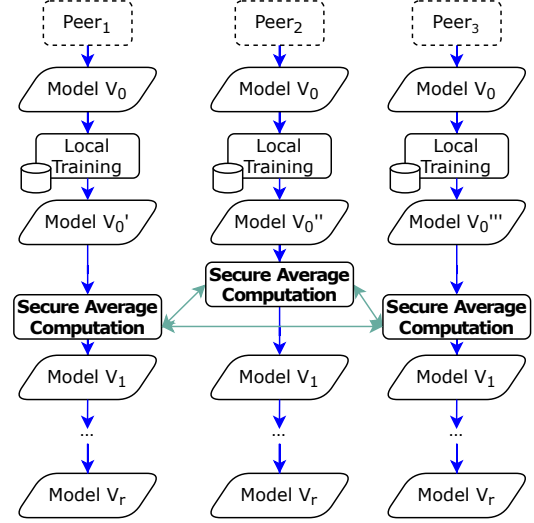


Fig. 1. Peer-to-peer model training with three peers

training datasets have been used by the respective peers. In the next phase, participants collaboratively conduct Secure Average Computation (SAC) steps to determine the average of corresponding model weight values in v'_0 , v''_0 and v'''_0 . During that process they do not exchange any model weight values among each other. As a result of SAC, all participants will possess identical updated model instances v_1 at their local premises. At this point participants may conduct local tests in order to decide, whether the new model version v_1 already has reached the expected quality, *e.g.*, prediction accuracy. If it is not yet the case, they may start another training round and further re-train their local v_1 instances while using again their respective local training datasets as input. It may take in total r rounds until the model v_r will converge, *i.e.*, hit the desired quality threshold, so that the training process can terminate.

From conceptual viewpoint, SAC is composed of a lightweight n -out-of- n secret partitioning scheme (see Algorithm III.1) and a secure multi-party average calculation method based on previously generated partitions (see Algorithm III.2).

To partition a given weight value w of a given neural-network model into n parts, the participant $Peer_i$ (as owner of w) first generates n positive random numbers $rn_1, \dots, rn_n$. These values will be divided by their sum, such that $prn_i = \frac{rn_i}{\sum_{k=1}^n rn_k}$, $i \in \{1, \dots, n\}$. The resulting percentage distribution is then used to generate normalized $w_1, \dots, w_n$ partitions of w , where $w_i = w * prn_i$, $i \in \{1, \dots, n\}$. For example, assume the weight $w=19$ has to be partitioned in three parts. The owner of w will first generate three random numbers, such as $rn_1=9$, $rn_2=18$, and $rn_3=30$ leading to the percentage factors of $prn_1=0.16$, $prn_2=0.32$ and $prn_3=0.52$, respectively. The resulting parts of w are thus $w_1=3$, $w_2=6$, and $w_3=10$ (see also in Table I).

Algorithm III.2 utilizes the partitioning method to calculate the mean of corresponding weight values of a neural-network

model in a distributed privacy-preserving manner. A schematic overview of the main communication and computation steps is shown in Figure 2. The algorithm makes use of the fact, that the sum S of n numbers can be calculated as the sum of subtotals of S , without explicitly knowing all n numbers.

Algorithm III.1 Algorithm to partition a secret value.

Require: n = number of peers; w = secret to be partitioned

```

1: function DIVIDE( $w, n$ )
2:    $rn$  = Array of  $n$  random numbers ( $rn_1, \dots, rn_n$ )
3:    $prn$  = Array of length  $n$  ( $prn_1, \dots, prn_n$ )
4:   for  $i = 1$  to  $n$  do
5:      $prn_i = \frac{rn_i}{\sum_{k=1}^n rn_k}$ 
6:   end for
7:    $par\_w$  = Array of length  $n$  ( $par\_w_1, \dots, par\_w_n$ )
8:   for  $i = 1$  to  $n$  do
9:      $par\_w_i = prn_i * w$ 
10:  end for
11:  return  $par\_w$ 
12: end function

```

Algorithm III.2 Multi-party average computation of a model weights tensor

Require: n = number of peers; wt = the weights tensor

```

1: function SECAVG( $n, wt$ )
2:    $par\_wt$  = a tensor with dimension  $dim(wt) + 1$ 
3:   for  $w \in wt$  do
4:      $par\_wt.append(Divide(w, n))$   $\triangleright$  Partition each weight into  $n$  parts.
5:   end for
6:   for  $i = 2$  to  $n$  do  $\triangleright$  Send a partition of each weight value to a peer, assuming that you are  $peer_1$ .
7:     Send  $par\_wt_i$  to  $peer_i$ 
8:   end for
9:   Save all received partitions and  $par\_wt_1$  in  $f\_wt$ 
10:   $ps\_wt$  = a tensor with dimension  $dim(wt) + 1$ 
11:  for  $f\_w \in f\_wt$  do  $\triangleright$  Calculate subtotals of each weight
12:     $ps\_wt.append(\sum_{i=1}^n f\_w_i)$ 
13:  end for
14:  Send  $ps\_wt$  to each peer
15:  Save all received subtotals and  $ps\_wt$  in  $s\_wt$ 
16:  for  $i = 1$  to  $|w|$  do  $\triangleright$  Totalize all subtotals received per weight and calculate the average
17:     $w_i = \frac{\sum_{j=1}^n s\_wt_{ij}}{n}$ 
18:  end for
19:  return  $w$ 
20: end function

```

After a given peer $Peer_i, i \in \{1, \dots, n\}$ has partitioned a model weight value w_i into the parts $w_{i1}, w_{i2}, \dots, w_{in}$, she or he provides each of the other $n - 1$ participants with a dedicated part w_{ij} ($i \neq j$) and keeps the remaining part w_{ii} secret. In the next step, $Peer_i$ determines the subtotal ps_i by summing up w_{ii} and the w_{ji} values that it received from the

TABLE I
SECURE AVERAGE COMPUTATION WITH THREE PEERS

	Peer1	Peer2	Peer3
Model weights	$w_1 = 25$	$w_2 = 19$	$w_3 = 37$
1.a) Initialize secret sharing	$rn_{11} = 10$ $prn_{11} = 0.2$ $rn_{12} = 20$ $prn_{12} = 0.4$ $rn_{13} = 20$ $prn_{13} = 0.4$	$rn_{21} = 9$ $prn_{21} = 0.16$ $rn_{22} = 18$ $prn_{22} = 0.32$ $rn_{23} = 30$ $prn_{23} = 0.52$	$rn_{31} = 30$ $prn_{31} = 0.41$ $rn_{32} = 20$ $prn_{32} = 0.27$ $rn_{33} = 24$ $prn_{33} = 0.32$
1.b) Create parts	$w_{11} = 5$ $w_{12} = 10$ $w_{13} = 10$	$w_{21} = 3$ $w_{22} = 6$ $w_{23} = 10$	$w_{31} = 15$ $w_{32} = 10$ $w_{33} = 12$
1.c) Share parts	$w_{21} = 3$ $w_{31} = 15$	$w_{12} = 10$ $w_{32} = 10$	$w_{13} = 10$ $w_{23} = 10$
2. Subtotals	$ps_1 = 23$	$ps_2 = 26$	$ps_3 = 32$
3. Totalization and averaging	$S = 81$ $Avg = 27$	$S = 81$ $Avg = 27$	$S = 81$ $Avg = 27$

other $n - 1$ peers. The locally computed subtotal ps_i will then be sent to all other participants. Upon receiving the subtotals from all peers, $Peer_i$ is able to compute S as the sum of subtotals and can calculate the average $Avg = S/n$ finally.

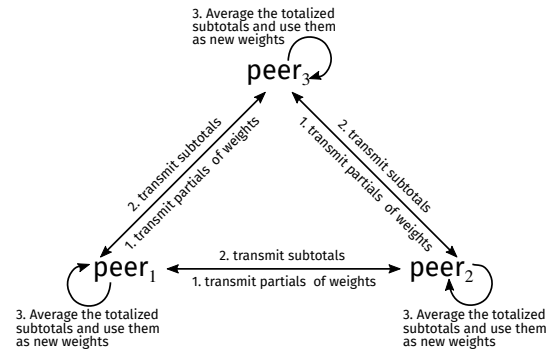


Fig. 2. Secure Average Computation with three peers

In Table I an example application case with $n=3$ participants $Peer_1$, $Peer_2$ and $Peer_3$ is given. The participants have previously trained the respective weight values w_1, w_2 and w_3 locally, using their own training datasets. As one can easily see, the mean of these values is 27.

The partitioning of w_i values using Algorithm III.1 results in the depicted w_{ij} parts. Each $Peer_i$ keeps the part w_{ii} secret and shares the other w_{ij} ($i \neq j$) parts with the other participants. For instance, $Peer_2$ sends w_{21} to $Peer_1$ and w_{23} to $Peer_3$. At the same time, $Peer_2$ receives the partitions w_{12} and w_{32} from $Peer_1$ and $Peer_3$, respectively. In the next step, a participant $Peer_i$ determines subtotal ps_i by summing up its own w_{ii} and the previously received w_{ji} values. $Peer_2$ calculates $ps_2=26$ as the sum of w_{22}, w_{12} and w_{32} . After the distribution of ps_i subtotals, each participant is able to calculate the average of the original w_i weight values. For instance, $Peer_2$ would receive ps_1 from $Peer_1$ and ps_3 from $Peer_3$. $Peer_2$ can now calculate the sum $S=ps_1+ps_2+ps_3$ and determine the average $Avg=S/n=27$.

IV. EXPERIMENTAL RESULTS

The main goal of experiments was to evaluate the basic applicability and technical feasibility of the approach in a distributed setup with five independent peers, each having separate disjoint training datasets. The experiments were conducted by using two relative “shallow” convolutional neural network (CNN) models that are typically used for image classification. These models were trained on publicly available and broadly used image datasets of modest size (MNIST and CIFAR-10).

In general, all experiments were performed with balanced data, that means all involved peers have used similarly large datasets in their respective local model training. With regard to feature distribution within the peer’s training datasets, three different cases were examined:

- **IID data:** Each peer’s training dataset is identically and independently distributed.
- **Non-IID data (5%):** Each peer’s training dataset has two main classes (out of ten), of which the given peer possesses by far the most training images. Only about 5% of the dataset represent the other (eight) remaining classes.
- **Non-IID data (0%):** Each peer’s training dataset consists only of the two main classes.

In all experiments the optimizer *Adam* was utilized, and as loss function categorical cross-entropy was taken. The number of epochs was identically set to 1, and the batch size was 64.

A. P2P Model Training using MNIST

The Modified NIST dataset [31] is a well-known set of images (28x28 pixels) showing handwritten digits. The dataset consists of a training set of 60,000 annotated images and a test set of 10,000 annotated images.

The neural-network model that were applied is a relatively small CNN. It consists of 8 layers and in total 1,663,370 weights. Figure 4 shows a schematic representation of the model. The layers are named in bold on the top. The values below the name describe the applied hyperparameters for the given layer. The corresponding tensor dimensions for each layer are shown at the bottom. In particular, this model was chosen because it was also used to train the MNIST dataset in [11], thus results should be comparable.

The goal of experiments with this dataset was to achieve a model accuracy of 99% with a learning rate of 0.001. This level of accuracy can be reached by training the model in a centralized setup, *i.e.*, without distributing training datasets among multiple entities at all. As one can see in figure 5, in the experiments using IID data this accuracy has been achieved already after 4 training rounds. In case of non-IID (5%) data, it took 155 training rounds. In the most extreme case with non-identically and independently distributed (IID) (0%) data the desired threshold was hit after 872 communication rounds.

In the setup with $n=5$ peers and $W=1,663,370$ weight values, the total communication effort per training round is to exchange about $2W(n^2-1)=66,534,800$ weight values over

the network, which corresponds to about 250 MB data in total, and thus approx. 50 MB per peer (assuming 32 bits per weight value).

B. P2P Model Training using CIFAR-10

The CIFAR-10 dataset [32] consists of 60,000 labeled RGB images (50,000 for training and 10,000 for testing) of 32x32 pixels in size. The model applied here is, again, a small CNN taken from a Keras example¹. Strictly speaking, it consists of 18 layers, since activation functions were modeled as individual layers. Figure 6 shows a schematic representation of the model. In this model there are two convolutional layers combined with a MaxPooling-layer each. Each of the MaxPooling-layers is followed by a Dropout-layer, and, there is one additional Dropout-layer before the Output-layer to avoid over-fitting. The four convolutional layers make this model in general more complex than the one depicted in Figure 4. However, this model has only 1,250,858 weights.

In experiments with this model and CIFAR-10 the target accuracy was set to 70%. This target was safely reached in experiments with the same model, but using the complete dataset. The learning rate was set in this case to 0.0001 and the training was limited to 500 rounds at the maximum. As shown in figure 7, the target of 70% accuracy could be met only with IID data: The threshold was reached after completing 220 training rounds. In case of non-IID (5%) data the aim was narrowly missed with about 68%. In the most extreme case with non-IID (0%) data there were observable improvements over time up to 44%. Note that this relatively low accuracy is still significantly higher than a peer could achieve alone using its own locally available non-IID (0%) data.

In this setup with $n=5$ peers and $W=1,250,858$ weights the total communication effort per training round is the exchange of about $2W(n^2-1)=50,034,320$ weights, *i.e.*, about 190 MB, and thus 38 MB per peer (assuming 32 bits per weight value).

V. ANALYSIS AND DISCUSSION

We characterize our approach regarding multiple aspects that have to be considered in potential applications:

- **Parallelism:** Within a training round each participant conducts all model training related calculations independent from others using own datasets. The training process is not entirely asynchronous, the SAC phase can be seen as the intermediate synchronization of locally determined ML model states. Nevertheless, the system ensures a good level of parallelism overall and can lead to faster training compared to rather sequential protocols, such as SL.
- **Anonymity:** In federated model training systems that rely on a trusted central server, usually only the server has to know and bi-laterally communicate with client nodes. Clients only know the identity of the trusted server, such as its address. They do not know whether and how many other clients are actually involved in the training procedure. As a consequence, a server can fool any client

¹<https://git.io/JqqUb>

being part of a large, anonymous client community and fetch the target client's data without being detected. In our approach peer group members are connected and actively communicate with each other, so that they could detect such manipulations.

- **Dealing with “semi-honest” participants:** A basic assumption is that peers honestly follow the protocol to jointly develop a common ML model, which they all may profit from. Nevertheless, some (all) of them may try to learn about each other's secret training datasets by analyzing messages they receive during regular protocol execution. This threat model is widely used in MPC [10] as well as in federated learning systems [33, 34]. Our approach equally protects single peers' training data against model reverse engineering [3] or membership inference attacks [4] that could be conducted by semi-honest peers, because locally computed model weights of particular peers are never shared in the system. A curious peer can only inspect aggregated ML models, which she or he co-determined using SAC. If the common ML model is highly over-fitted, for example, because peers possess very similar datasets, the likelihood of successful training data determination increases [4]. However, even a perfect match would not necessarily reveal the actual owner of the given input data item. In a peer-group with size of $n = 2$, both participants could easily calculate each other's model weights based on the final average values and determine not only input data items but also its owner. Therefore, a peer group must not have less than three members during the protocol execution.

It is worth noting that the secret sharing schema does not hide the sign of partitioned model weights. Thus, each peer knows if and in which round the sign of a certain weight value was changed during local training steps. We can see further information leakage in a special case: If $Peer_i$ partitions a local model weight $w_i=0$, for all parts $w_{ij}=0$ will hold, and thus all other peers can trivially determine w_i .

In addition to “semi-honest” participants a peer group can have actively malicious members that do not follow the protocol. Such members could send around fake values during SAC steps leading to poor quality of the resulting common model. This kind of sabotage could be detected by other peers, but it would be a difficult task to figure out who is actually breaking the rules of the game.

- **Communication effort:** When using SAC the communication cost c per training round is estimated as follows. In the first step, each of n peers sends the computed partitions of W model weight values to the other $n-1$ peers. This results in a cost of $c_1 = n*(n-1)*W$. In the second step, the calculated subtotals for each of the W weight values are sent to $n-1$ participants, again incurring a cost of $c_2 = n*(n-1)*W$. Therefore, the total cost per training round adds up to $c = c_1 + c_2 = 2W(n^2 - n)$. Note that without using SAC all n peers would have to share all W weights with $n-1$ peers to compute averages,

leading to an effort $c = W * (n^2 - n)$ per round. The number of actually transmitted bytes depends on specific data types used to encode W weights. In a practical implementation participants will rather exchange files, instead of sending messages *e.g.*, per weight value. The overall communication cost C to train a common model will significantly depend on the number r of required training rounds as $C = r * c$ holds. As shown in our experiments, r is mainly determined by the actual feature distribution of training data.

- **Impact of rounding errors:** In typical ML model implementations floating-point data types are used to encode weights. Therefore, rounding errors may occur while partitioning weight values during SAC, which has no major negative impact on the performance of resulting common models.
- **Robustness:** A peer's unintended exit during an ongoing training round, *e.g.*, due to network or system failures, would jeopardize training results. Peers may individually detect such problems, *e.g.*, when a pre-configured timeout was elapsed, and restart the training without the missing participant. Before restarting the training, participants have to check whether the minimum required number of peers are alive.

VI. CONCLUSION AND OUTLOOK

Training experiments with example neural networks on well-known sample datasets have proven the basic technical feasibility of our peer-to-peer approach. Even under extreme feature distribution of training data among participants, resulting ML models have achieved a high level of prediction accuracy.

Compared to server-based model training systems, the presented peer-to-peer approach is certainly not the best available choice in application scenarios with thousands or even millions of participants mainly due to the inherently higher communication costs. In smaller peer groups, such as of companies competing in the same business niche, it can be a suitable way to indirectly benefit from each other's datasets without compromising mission-critical information. Future research will nevertheless focus on the optimization of communication efforts, *i.e.*, reducing the required number of training rounds to reach a targeted model accuracy.

Currently it is under evaluation whether the mainly privacy-related advantages can make the approach attractive enough for companies to establish and run peer groups for collaborative ML model training. In a commercial setup, a service provider might act as a “technical orchestrator” to administer peer groups, manage the on- and off-boarding of companies, help them create and connect training pipelines, distribute proper parameter values for model training, etc. The design and implementation of a corresponding technical framework for operating the overall system is therefore subject of future work as well.

REFERENCES

- [1] Qiang Yang et al. “Federated Machine Learning: Concept and Applications”. In: *ACM Trans. Intell. Syst. Technol.* 10.2 (Jan. 2019).
- [2] Jakub Konečný, Brendan McMahan, and Daniel Ramage. *Federated Optimization: Distributed Optimization Beyond the Datacenter*. 2015. arXiv: 1511.03575.
- [3] Ulrich Aïvodji, Sébastien Gambs, and Timon Ther. *GAMIN: An Adversarial Approach to Black-Box Model Inversion*. 2019. arXiv: 1909.11835.
- [4] Reza Shokri et al. *Membership Inference Attacks against Machine Learning Models*. 2017. arXiv: 1610.05820 [cs.CR].
- [5] Cynthia Dwork. “Differential Privacy”. In: *Automata, Languages and Programming*. Ed. by Michele Bugliesi et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 1–12.
- [6] H. Brendan McMahan et al. “Learning Differentially Private Language Models Without Losing Accuracy”. In: *CoRR* (2017). arXiv: 1710.06963.
- [7] Abhishek Bhowmick et al. *Protection Against Reconstruction and Its Applications in Private Federated Learning*. 2018. arXiv: 1812.00984.
- [8] Robin C. Geyer, Tassilo Klein, and Moin Nabi. *Differentially Private Federated Learning: A Client Level Perspective*. 2018. arXiv: 1712.07557.
- [9] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. 7th ed. Boston, MA: Pearson, 2016.
- [10] David Evans, Vladimir Kolesnikov, and Mike Rosulek. “A Pragmatic Introduction to Secure Multi-Party Computation”. In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246.
- [11] H. Brendan McMahan et al. “Federated Learning of Deep Networks using Model Averaging”. In: *CoRR* (2016). arXiv: 1602.05629.
- [12] H. Brendan McMahan et al. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. 2016. arXiv: 1602.05629.
- [13] Jakub Konečný et al. “Federated Learning: Strategies for Improving Communication Efficiency”. In: *CoRR* (2016). arXiv: 1610.05492.
- [14] Jakub Konečný et al. “Federated Optimization: Distributed Machine Learning for On-Device Intelligence”. In: *CoRR* (2016). arXiv: 1610.02527.
- [15] Y. Chen, X. Sun, and Y. Jin. “Communication-Efficient Federated Deep Learning with Asynchronous Model Update and Temporally Weighted Aggregation”. In: *CoRR* (2019). arXiv: 1903.07424.
- [16] Andrew Hard et al. “Federated Learning for Mobile Keyboard Prediction”. In: *CoRR* (2018). arXiv: 1811.03604.
- [17] R. Bekkerman, M. Bilenko, and J. Langford, eds. *Scaling Up Machine Learning*. Cambridge University Press, 2009.
- [18] H. Brendan McMahan and Galen Andrew. “A General Approach to Adding Differential Privacy to Iterative Training Procedures”. In: *CoRR* (2018). arXiv: 1812.06210.
- [19] Cynthia Dwork et al. “Calibrating Noise to Sensitivity in Private Data Analysis”. In: *Theory of Cryptography*. Springer Berlin Heidelberg, 2006, pp. 265–284.
- [20] Galen Andrew, Steve Chien, and Nicolas Papernot. *tensorflow/privacy: Library for training machine learning models with privacy for training data*. 2019. URL: <https://github.com/tensorflow/privacy>.
- [21] Sashank Reddi et al. *Adaptive Federated Optimization*. 2020. arXiv: 2003.00295.
- [22] O. Gupta and R. Raskar. “Distributed learning of deep neural network over multiple agents”. In: *Journal of Network and Computer Applications* (2018), pp. 1–8.
- [23] Maarten G. Poiriot et al. *Split Learning for collaborative deep learning in healthcare*. 2019. arXiv: 1912.12115.
- [24] P. Vepakomma et al. “Reducing Leakage in Distributed Deep Learning for Sensitive Health Data”. In: 2019.
- [25] Abhijit Guha Roy et al. “BrainTorrent: A Peer-to-Peer Environment for Decentralized Federated Learning”. In: *CoRR* abs/1905.06731 (2019). arXiv: 1905.06731.
- [26] Aurélien Bellet et al. “Personalized and Private Peer-to-Peer Machine Learning”. In: *Proc. of the 21st Intl. Conf. on Artificial Intelligence and Statistics*. Ed. by A. Storkey and F. Perez-Cruz. Vol. 84. Proc. of Machine Learning Research. PMLR, 2018, pp. 473–481.
- [27] Amin Fadaeddini, Babak Majidi, and Mohammad Esghi. “Secure decentralized peer-to-peer training of deep neural networks based on distributed ledger technology”. In: *The Journal of Supercomputing* 76 (Dec. 2020).
- [28] Adi Shamir. “How to share a secret”. In: *Communications of the ACM* 11 (1979), pp. 612–613.
- [29] G. R. Blakley. “Safeguarding cryptographic keys”. In: *Managing Requirements Knowledge, International Workshop on*. Los Alamitos, CA, USA: IEEE Computer Society, 1979, p. 313.
- [30] Eleftheria Makri. *CO6GC: LINEAR SECRET SHARING SCHEMES – LSSS – COSIC*. Apr. 17, 2020. URL: <https://www.esat.kuleuven.be/cosic/blog/lsss/>.
- [31] Y. LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 11 (1998), pp. 2278–2324.
- [32] A. Krizhevsky, G. Hinton, et al. “Learning multiple layers of features from tiny images”. In: (2009).
- [33] Keith Bonawitz et al. “Practical secure aggregation for privacy-preserving machine learning”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 1175–1191.
- [34] Stacey Truex et al. *A Hybrid Approach to Privacy-Preserving Federated Learning*. 2019. arXiv: 1812.03224.

APPENDIX

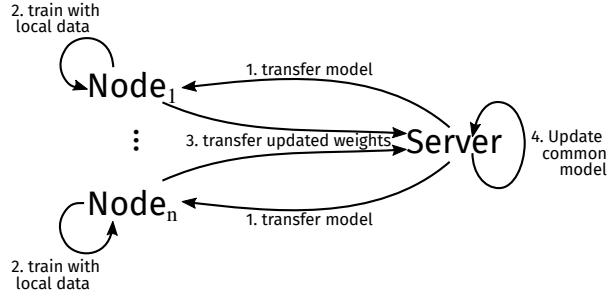


Fig. 3. Federated optimization using a central server

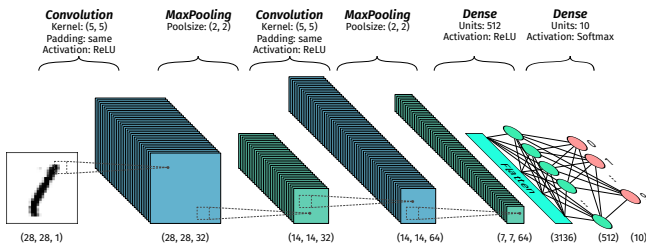


Fig. 4. CNN model trained on MNIST dataset.

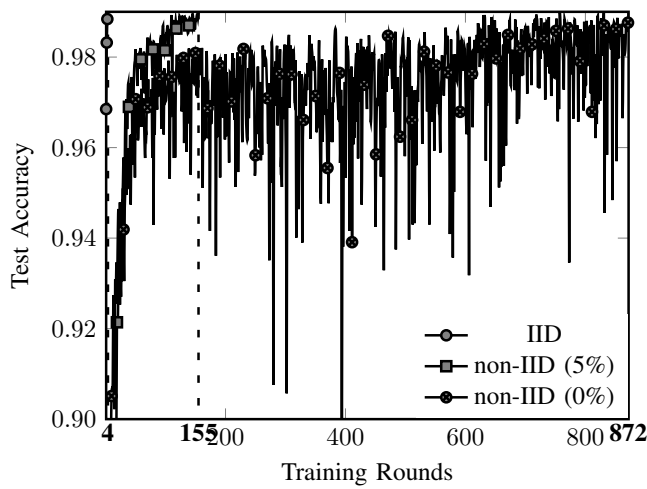


Fig. 5. Development of accuracy with P2P training on MNIST

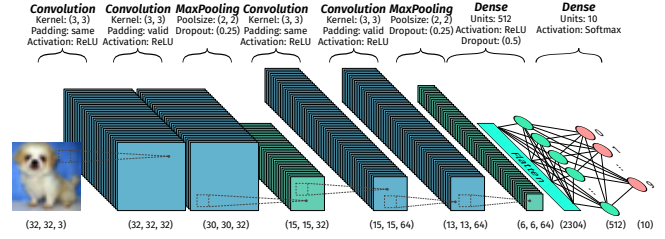


Fig. 6. CNN model trained on CIFAR-10 dataset

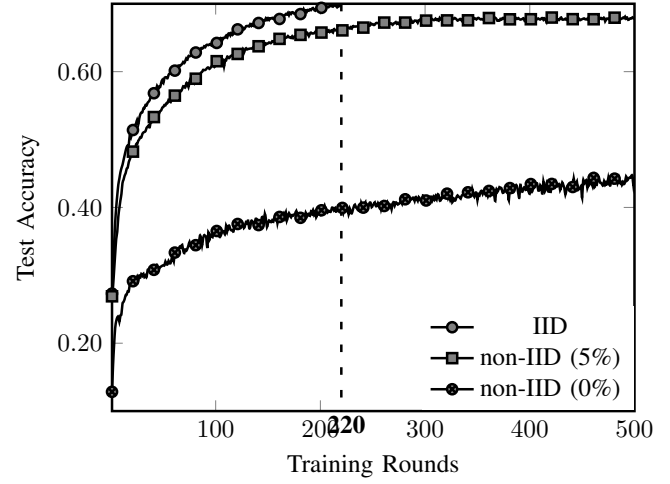


Fig. 7. Development of accuracy with P2P training on CIFAR-10